

pGM-JAX: differentiable polarizable Gaussian multipole electrostatics and molecular dynamics for force-field parameterization

[Author 1]¹, [Author 2]¹, [Author 3]¹, and [Corresponding author]^{*1}

¹[Affiliation]

Abstract

Polarizable force fields are harder to parameterize than fixed-charge ones because every observable depends on self-consistent induced dipoles. We present pGM-JAX, an implementation of the polarizable Gaussian multipole (pGM) model in JAX in which energies, forces, induced dipoles and virials are differentiable functions of the coordinates, the periodic box and every parameter. The package contains gas-phase and Ewald models for fitting to quantum-chemical data, and a GPU molecular dynamics engine built on JAX MD. The engine uses smooth particle-mesh Ewald for Gaussian charges and dipoles, the induced-dipole solver of Amber’s pmemd-pgm, rigid or flexible molecules and NVE, Langevin NVT and Monte Carlo NPT ensembles. Induced dipoles are differentiated implicitly, so a derivative of forces or dipoles costs one extra linear solve. Against Amber, electrostatic energies of a 512-water box agree to 8×10^{-11} relative, forces to 9×10^{-7} kcal mol⁻¹ Å⁻¹ RMS and induced dipoles to 10^{-10} e Å. On one GPU the engine runs this box at 130 ns/day, compared with 96 ns/day for pmemd-pgm’s CUDA code, and 4,096 waters at 44 ns/day (pmemd-pgm: 69). Derivatives of ensemble averages follow from fluctuation formulas with per-frame parameter gradients taken at the converged dipoles. We use them to fit Lennard-Jones parameters to the density and heat of vaporization of liquid water and methanol by Gauss–Newton iteration, one simulation per step. Bonded terms fitted to DFT data make molecules flexible in the same engine. The repository contains the code, the tests, the data behind every figure and step-by-step workflows for bonded and van der Waals parameterization.

1 Introduction

Molecules polarize in response to their environment, and polarizable force fields describe this explicitly, usually with induced point dipoles [1, 2]. Point dipoles need damping at short range [3] and exclusion rules for bonded pairs. The polarizable Gaussian multipole (pGM) model replaces point charges and dipoles by Gaussian distributions [4–6]. The interaction between two Gaussians is finite at any distance, so every pair of atoms interacts, including bonded ones, and the model has no exclusions and no separate damping function. Permanent dipoles are defined along covalent bonds, so they follow the molecular geometry without local frames [6]. Polarizabilities and Gaussian widths are typed by element and environment [5]; charges and covalent dipoles are fitted to quantum-chemical electrostatic potentials with PyRESP [7, 8] and transfer between molecules [9]. pGM is implemented in Amber’s sander and pmemd, including the stress tensor for constant-pressure simulation [10–12], and pGM water models have been refined against liquid properties [13].

The remaining parameters (Lennard-Jones or other van der Waals terms, and bonded terms for flexible molecules) are fitted to condensed-phase properties and to quantum-chemical energies

^{*}[email]

and forces. For a polarizable model this is expensive: each liquid simulation solves for the induced dipoles at every step, and derivatives of simulated properties with respect to parameters are rarely available, so parameters are often tuned by hand or by black-box search [13]. ForceBalance showed that thermodynamic fluctuation formulas give these derivatives from a single simulation and made systematic optimization of polarizable water models possible [14, 15]. Automatic differentiation removes most of the remaining work: with JAX [16] or PyTorch, derivatives of any energy function come for free. JAX MD [17], TorchMD [18], DiffTRe [19] and DMFF [20] use this for differentiable simulation and force-field development; DMFF includes multipolar polarizable models.

pGM-JAX is a differentiable implementation of pGM itself, meant as the working tool for pGM parameterization. Its design goals are:

1. *Exactly Amber’s model.* The same energy as sander and pmemd-pgm to numerical precision, so parameters fitted in pGM-JAX can be used in Amber and vice versa.
2. *Parameters are inputs.* Every energy function takes the parameters as an argument, with user-defined tying of atoms that share a parameter, so gradients with respect to any subset of parameters come from `jax.grad`.
3. *Derivatives through the induction.* First derivatives use the stationarity of the energy in the induced dipoles; forces, dipoles and higher derivatives are differentiated implicitly through the dipole solve.
4. *Fast enough to sample liquids.* A GPU MD engine that runs pGM at the speed of Amber’s CUDA code for small and medium boxes, so that a fitting iteration is one short simulation.

This paper describes the model and its implementation (Sections 2 and 3), validation against Amber (Section 4), performance (Section 5), and two demonstrations: flexible molecules in MD and fitting of Lennard-Jones parameters to liquid properties (Section 6). Section 7 describes the workflows for fitting bonded and van der Waals parameters.

2 Model

Electrostatics. Atom i carries a Gaussian charge q_i , a permanent dipole $\mathbf{p}_i$ and an induced dipole $\boldsymbol{\mu}_i$, all with the Gaussian width of the atom, R_i . The permanent dipole is a sum over covalent bonds, $\mathbf{p}_i = \sum_{j \in \text{cov}(i)} c_{ij} \hat{\mathbf{r}}_{ij}$, with $\hat{\mathbf{r}}_{ij}$ the unit vector from i to j and c_{ij} a fitted strength, so the permanent dipoles follow the molecular geometry [6]. Two Gaussians of widths R_i and R_j interact through

$$\phi_{ij}(r) = \frac{\text{erf}(\beta_{ij}r)}{r}, \quad \beta_{ij} = [2(R_i^2 + R_j^2)]^{-1/2}, \quad (1)$$

which is finite at $r = 0$. The permanent energy is a sum over *all* pairs,

$$U_{\text{perm}} = k_e \sum_{i < j} (q_i + \mathbf{p}_i \cdot \nabla_i)(q_j + \mathbf{p}_j \cdot \nabla_j) \phi_{ij}(|\mathbf{r}_i - \mathbf{r}_j|), \quad (2)$$

where k_e is the Coulomb constant and ∇_i acts on $\mathbf{r}_i$. The induced dipoles minimize

$$U_{\text{ind}}(\boldsymbol{\mu}) = \frac{1}{2} \boldsymbol{\mu}^\top \mathbf{A} \boldsymbol{\mu} - \boldsymbol{\mu}^\top \mathbf{E}, \quad \mathbf{A} = \boldsymbol{\alpha}^{-1} + \mathbf{T}, \quad (3)$$

with $\boldsymbol{\alpha}$ the diagonal matrix of isotropic atomic polarizabilities, $\mathbf{E}$ the field of the permanent Gaussian multipoles and $\mathbf{T}$ the Gaussian dipole–dipole tensor ($\mathbf{T}_{ij} = k_e \nabla_i \nabla_j \phi_{ij}$ for $i \neq j$). At the minimum $\mathbf{A} \boldsymbol{\mu}^* = \mathbf{E}$ and $U_{\text{ind}}^* = -\frac{1}{2} \boldsymbol{\mu}^{*\top} \mathbf{E}$. Because $\partial U / \partial \boldsymbol{\mu} = 0$ at $\boldsymbol{\mu}^*$, first derivatives of the energy with respect to positions, box or parameters need no derivative of $\boldsymbol{\mu}^*$ (Hellmann–Feynman). Periodic systems are summed by Ewald methods; the Gaussian widths enter the real-space kernel only.

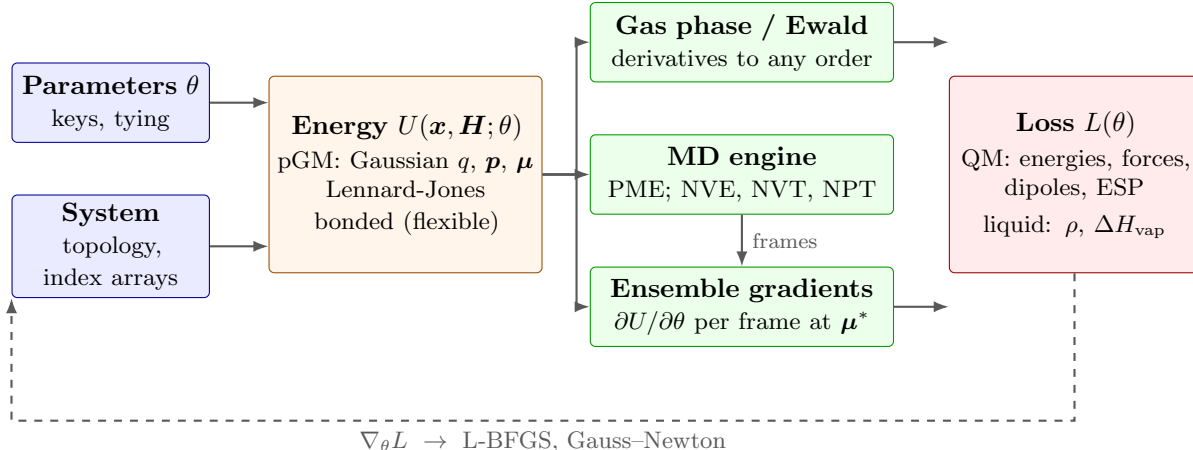


Figure 1: Structure of pGM-JAX. Parameters are an explicit input of every energy function; gradients of a loss with respect to them come from automatic differentiation of the gas-phase or periodic models, or from fluctuation formulas over MD frames.

Van der Waals. Lennard-Jones in Amber form, $U_{\text{LJ}} = \sum \varepsilon_{ij} [(R_{ij}^*/r)^{12} - 2(R_{ij}^*/r)^6]$ with $R_{ij}^* = R_i^* + R_j^*$ and $\varepsilon_{ij} = \sqrt{\varepsilon_i \varepsilon_j}$, optionally with the long-range tail correction. The free parameters are R_i^* and $\sqrt{\varepsilon_i}$ (the square root keeps the gradient finite at $\varepsilon_i = 0$, as for water hydrogens). For rigid molecules Lennard-Jones acts between molecules only.

Flexible molecules. Electrostatics already includes every intramolecular pair, so the valence terms of a flexible pGM molecule only have to carry what the all-pair electrostatics and the Lennard-Jones between atoms far apart in the bond graph do not:

$$E_{\text{intra}} = E_{\text{bonded}}(\mathbf{R}) + \sum_{i < j, d_{ij} \geq n_{\text{LJ}}} U_{ij}^{\text{LJ}} + s_{14} \sum_{d_{ij}=3} U_{ij}^{\text{LJ}}, \quad (4)$$

with d_{ij} the number of bonds between i and j , $n_{\text{LJ}} = 4$ by default (Lennard-Jones from 1–5 pairs on) and an optional scaled 1–4 term. E_{bonded} is a sum of term families (Section 3.5) fitted to quantum-chemical energies and forces with exactly this nonbonded model, so the same energy is used in fitting and in MD.

3 Implementation

Figure 1 shows the structure of the package. Everything is written in JAX and runs on CPUs and GPUs; float64 is used throughout except in the MD engine’s mixed-precision mode.

3.1 Parameters as inputs

A `Molecule` holds a topology and initial parameter values (read from an Amber pGM prmtop or PyRESP output). A `ParamTable` turns them into free parameters: for each quantity (q , covalent-dipole strength c , width R , polarizability α , R^* , $\sqrt{\varepsilon}$) a list of keys, one value per key. Atoms with the same key share one value and one gradient. By default symmetry-equivalent atoms of a molecule share charges and dipole strengths (as in PyRESP), found by colour refinement on the bond graph and the covalent-dipole pairs, and R , α and the Lennard-Jones parameters are shared by atom type;

any other tying is set by giving keys. One table built from several molecules ties parameters across them. The parameter pytree is an argument of every energy function, so `jax.grad` returns a pytree of the same shape:

```
sys = System([water] * 3)                # topology + index arrays into the table
P = sys.table.initial()                  # {"q": ..., "cov": ..., "lj_sqrt_eps": ...}
E = Model([ElecChannel(), LJChannel()]).energy_fn(sys)
dE_dP = jax.grad(lambda p: E(pos, p)["total"])(P)          # same pytree as P
F = Model([ElecChannel(), LJChannel()]).forces_fn(sys)
dL_dP = jax.grad(lambda p: jnp.sum((F(pos, p) - F_qm) ** 2))(P)  # force matching
```

3.2 Differentiating through the induced dipoles

In the gas phase the dipoles come from a dense solve of $\mathbf{A}\boldsymbol{\mu} = \mathbf{E}$ (up to a few thousand atoms); periodic single-point models use Ewald summation with integer image and k -vector lists, so the box matrix is a differentiable input, and solve by conjugate gradients. The energy is wrapped so that first derivatives use stationarity and higher derivatives differentiate through the solve by implicit differentiation [21]. Differentiating $\mathbf{A}\boldsymbol{\mu}^* = \mathbf{E}(\theta)$ gives $\mathbf{A}d\boldsymbol{\mu}^* = d\mathbf{E} - d\mathbf{A}\boldsymbol{\mu}^*$, so a vector–Jacobian product with $\bar{\boldsymbol{\mu}}$ needs one adjoint solve,

$$\mathbf{A}\boldsymbol{\lambda} = \bar{\boldsymbol{\mu}}, \quad \bar{\boldsymbol{\mu}} \cdot \frac{d\boldsymbol{\mu}^*}{d\theta} = \boldsymbol{\lambda} \cdot \frac{\partial(\mathbf{E} - \mathbf{A}\boldsymbol{\mu})}{\partial\theta} \Big|_{\boldsymbol{\mu}^*}, \quad (5)$$

with the same symmetric operator. We checked that this matters: a deliberately broken version that treats $\boldsymbol{\mu}^*$ as a constant gives second-order parameter derivatives that are wrong by 12–100 %, while the implicit version agrees with finite differences (Section 4).

3.3 Molecular dynamics engine

The MD engine (`pgm_jax.md`) follows `pmemd-pgm` [6, 10, 11] and uses JAX MD [17] for cell lists and rigid-body integration.

Electrostatics. Smooth PME [22] for Gaussian charges and total dipoles ($\mathbf{p} + \boldsymbol{\mu}$), spread with B-spline derivatives. The real-space part runs over “rows”: each atom’s intramolecular partners, then its intermolecular neighbours, compacted every step to the pairs inside the cutoff and stored as a structure of arrays, with per-atom sums and analytic pair forces instead of scatter-adds. Lennard-Jones uses the same rows, with an optional long-range tail correction.

Neighbour lists. For rigid molecules the list is built between molecular centres with JAX MD’s cell list; rotations never invalidate it, so for water it is rebuilt about every 30 steps instead of every 10 for an atom list, at a tenth of the cost. Every overflow of a list or of the row capacity is detected, the capacities are enlarged and the block of steps is repeated. When the box volume drifts by more than 10 % (NPT from a loose start) or a block keeps overflowing, the lists are rebuilt for the new box and the block is split.

Induced dipoles. As in `pmemd-pgm`’s GPU code: the initial guess is a cubic extrapolation from the dipoles of the last four steps, the first residual is fused with the permanent-field evaluation, and a Jacobi-preconditioned conjugate-gradient solve runs to `pmemd-pgm`’s convergence criterion ($\max|\alpha r|/\max|\alpha E| \leq \text{dipole_scf_tol}$, default 10^{-5}), followed by a “peek” step. Forces are evaluated at the converged dipoles.

Integrators. Rigid molecules are JAX MD rigid bodies integrated with the NO_SQUISH quaternion scheme [23]. Flexible molecules are integrated atom by atom with the bonded energy of Eq. (4), evaluated for all molecules of a type with one vectorized call, and are kept whole across

periodic boundaries. Temperature is controlled by BAOAB Langevin dynamics [24] with an exact Ornstein–Uhlenbeck step (on centre-of-mass and body-frame angular momenta for rigid molecules), pressure by an isotropic Monte Carlo barostat [25, 26] with molecular scaling and an adaptive step. Two issues of JAX MD 0.2.29 are worked around: its rigid-body Langevin step draws the quaternion-momentum noise with a diagonal covariance, which leaves water rotations 15–25 K too cold, and its neighbour list flags valid box matrices as malformed.

Precision and I/O. In mixed precision, pair kernels, PME and CG vectors are float32 and positions, energies and reductions float64; float32 matrix products are requested at full precision because the default TF32 mode of NVIDIA GPUs made the PME dipole spreading 10^{-3} inaccurate. Any reduced triclinic box is supported. Inputs are Amber prmtop and inpcrd/rst7 files; trajectories and restarts are Amber NetCDF, and checkpoints continue a run exactly up to floating-point summation order.

Differentiable forces and dipoles. With `differentiable=True` the force routine returns energy, forces and induced dipoles that can be differentiated with respect to parameters, positions and box, through a custom vector–Jacobian product that implements Eq. (5) with one extra CG solve. The forward pass is unchanged, so MD runs at the same speed with the option on, and force or dipole matching on periodic frames is a few lines:

```
ff = PGMForceField(system, H, MDSettings(precision="double", differentiable=True))
rows = ff.rows_for(pos, H) # once per frame, on the host
def loss(theta):
    res = ff.compute(pos, H, rows, ff.init_induction(), theta)
    return jnp.sum((res.forces - F_ref) ** 2) + w * jnp.sum((res.induction.mu - mu_ref) ** 2)
g = jax.jit(jax.grad(loss))(system.params0)
```

3.4 Derivatives of ensemble averages

Trajectories themselves are not differentiated: neighbour-list rebuilds, Monte Carlo moves and the Langevin noise make that neither possible nor useful. For an NPT ensemble at parameters θ and an observable $A(\mathbf{x}, V; \theta)$,

$$\frac{d\langle A \rangle}{d\theta} = \left\langle \frac{\partial A}{\partial \theta} \right\rangle - \beta \left(\left\langle A \frac{\partial U}{\partial \theta} \right\rangle - \langle A \rangle \left\langle \frac{\partial U}{\partial \theta} \right\rangle \right), \quad (6)$$

as used in ForceBalance [14]. The only model-specific quantity is $\partial U / \partial \theta$ for each saved frame. Because the pGM energy is stationary in the induced dipoles, it is the partial derivative at fixed converged dipoles, one reverse-mode pass through the energy (`energy.fixed_mu`), with no derivative of the dipole solve. For the density $\rho = M/V$ and the heat of vaporization $\Delta H_{\text{vap}} = \langle U_{\text{gas}} \rangle - \langle U_{\text{liq}} \rangle / N + RT$, Eq. (6) gives the Jacobian of both targets with respect to any set of parameters from one liquid simulation. For molecules without intramolecular Lennard-Jones pairs (rigid molecules, and the flexible methanol below) $\langle U_{\text{gas}} \rangle$ does not depend on the Lennard-Jones parameters; otherwise Eq. (6) applies to gas-phase frames as well.

3.5 Bonded terms

Because pGM has no exclusions, the bonded terms of a flexible pGM molecule differ from those of fixed-charge force fields such as GAFF [27]: they correct an all-pair electrostatic energy instead of replacing a masked one. `pgm_jax.bonded` provides the topology (internal coordinates and coupling index sets from the bond graph), a registry of term families (the class II set [28] used by Abdullah et al. [29], with Morse bonds, cosine angles, Fourier torsions and bond–bond, bond–angle, angle–angle, torsion–bond, torsion–angle and angle–angle–torsion couplings, plus Urey–Bradley and other

Table 1: pGM-JAX against Amber for 512 pGM waters: differences unless stated otherwise. Energies in kcal/mol, forces in kcal mol⁻¹ Å⁻¹, dipoles in *e* Å.

Comparison	Result
Gas-phase cluster vs sander (no cutoff)	electrostatic energy 2×10^{-5} of $-505,230$; forces 4.2×10^{-5} RMS (RMS force 38.8); LJ 623.8568, identical
Periodic vs pmemd-pgm (PME 72 ³ , order 8, float64)	electrostatic energy 4×10^{-5} of $-506,165$; forces 8.8×10^{-7} RMS; induced dipoles of every atom 1.2×10^{-10} RMS; LJ 690.1238, identical
Same, MD engine in mixed precision	electrostatic energy 0.05; forces 4×10^{-4} RMS (RMS force 36); dipoles 8×10^{-7} RMS
Molecular virial vs sander (<i>ntp</i> = 1), no LJ tail	547.49803 vs 547.498 (values)
Same, with LJ tail correction	610.73566 vs 610.7357 (values)
Water monomer vs PyRESP	induced dipoles to 8×10^{-10} a.u.
NPT, 298 K, 1 bar (8 Å cutoff, PME 48 ³ , order 8, no LJ tail, <code>dipole_scf_tol</code> 10^{-4}), 2 runs of 100 ps	density 1.0058 ± 0.011 and 1.0107 ± 0.012 g cm ⁻³ (pmemd-pgm, same settings: 1.0076 ± 0.012); first peak of <i>g</i> _{OO} at 2.79 Å, height 3.00 (pmemd-pgm: 2.795 Å, 2.99)

topological pair terms, out-of-plane terms and others; a new family is about 20 lines), the model of Eq. (4) with gas-phase pGM, and an L-BFGS fitter to energies, forces and dipoles. Reference data are frames sampled with MACE-OFF [30] and labelled at the ω B97M-D3(BJ)/def2-TZVPPD level [31–34] with Psi4 [35]. A study of which functional forms work best with all-pair Gaussian electrostatics will be reported separately; here the fitted parameters are exported as a `FlexibleTemplate` for the MD engine (Section 6.1).

4 Validation

Table 1 compares pGM-JAX with Amber for a box of 512 rigid three-site pGM waters with Lennard-Jones on oxygen. Amber’s pGM code uses a Coulomb constant (332.05382 kcal Å mol⁻¹ *e*⁻²) that is 2.98×10^{-5} smaller than the CODATA value used by pGM-JAX; comparisons rescale by this ratio.

Derivatives are tested against finite differences in the test suite (`pytest`, 51 tests, 4 minutes on a CPU node): analytic real-space row forces against automatic differentiation and finite differences (2×10^{-10} relative in float64); forces and the virial (the derivative with respect to the box) against finite differences; derivatives of forces and induced dipoles with respect to parameters, positions and box against finite differences with the dipoles re-solved (10^{-7} to 10^{-10} relative in float64; mixed-precision gradients agree with float64 to 10^{-6} to 5×10^{-5}); and second derivatives of the energy with respect to parameters. Other tests compare PME with exact Ewald sums, check that the neighbour list is complete in skewed boxes, equipartition between translation and rotation under the Langevin thermostat, NVE energy conservation, exact restarts and the flexible-molecule engine (Section 6.1).

In NVE with mixed precision and a 1 fs step, the energy drift of 512 rigid waters over 20 ps is 10^{-4} *kT*/ns per degree of freedom at `dipole_scf_tol` = 10^{-5} and 0.02 *kT*/ns at 10^{-4} . Under Langevin dynamics ($\gamma = 2$ ps⁻¹) the temperature is 297–299 K with equal translational and rotational temperatures.

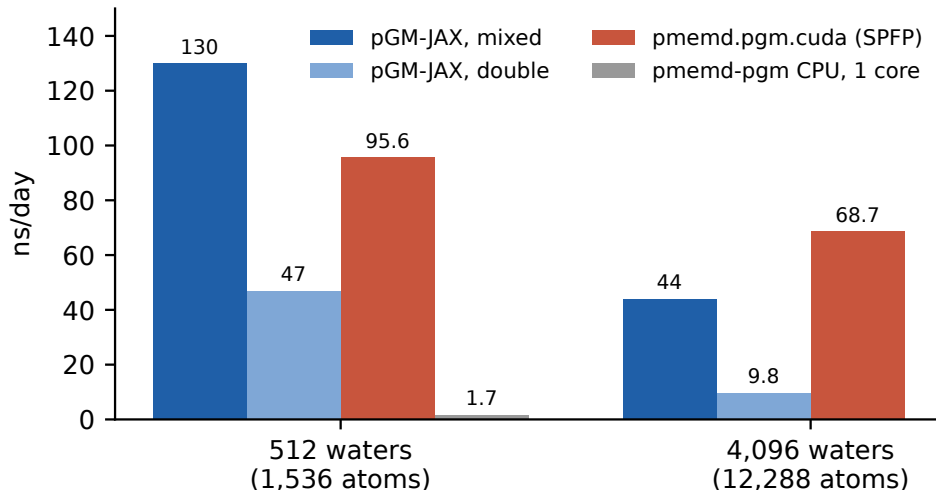


Figure 2: MD speed on one NVIDIA RTX PRO 6000 Blackwell GPU (pmemd-pgm CPU: one core). NVT, $\gamma = 1 \text{ ps}^{-1}$, $9 \text{ \AA}$ cutoff, PME grid 48^3 per 512 waters, order 6, 1 fs step, `dipole_scf_tol` = 10^{-5} .

5 Performance

Figure 2 compares the MD engine with pmemd-pgm under identical settings. For 512 waters (1,536 atoms) pGM-JAX runs at 130 ns/day in mixed precision (47 in double), against 95.6 ns/day for `pmemd.pgm.cuda_SPFP` and 1.7 ns/day for pmemd-pgm on one CPU core. For 4,096 waters (12,288 atoms) it runs at 44 ns/day (9.8 in double) against 68.7. With `dipole_scf_tol` = 10^{-4} the speeds are 148 and 53 ns/day (1.63 ms per step for 12k atoms). The step became fast through the molecular-centre neighbour list (list cost per step from about 1 ms to 0.03 ms at 12k atoms), row storage as a structure of arrays (the row kernels are memory bound), a direct PME dipole gradient inside the CG iteration instead of automatic differentiation, closed-form 3×3 box inverses, and the fused residual. The remaining gap to pmemd-pgm at 12k atoms is hand-written CUDA against XLA: pmemd-pgm runs one fused kernel per term and CG iteration, XLA several kernels with the loop condition checked on the host. A gradient of forces and dipoles with respect to all parameters costs about 15 force evaluations (30 ms for 12k atoms).

6 Demonstrations

6.1 Flexible molecules in MD

We tested the flexible-molecule engine on liquid methanol. The bonded terms are the class II set (Morse bonds, cosine angles, Fourier torsions, an improper term and bond-bond, bond-angle, angle-angle, torsion-bond, torsion-angle and angle-angle-torsion couplings), fitted with the all-pair pGM model of Eq. (4) to DFT energies and forces of 224 frames (MACE-OFF molecular dynamics at 500 K and relaxed torsion scans). On 400 test frames sampled at 298 K they give an energy error of 0.13 kcal/mol (mean absolute, after removing the mean offset) and a mean force error of $2.3 \text{ kcal mol}^{-1} \text{ \AA}^{-1}$ per atom. The pGM charges and covalent dipoles come from PyRESP, the Lennard-Jones parameters from GAFF. Methanol has no pairs more than three bonds apart, so Eq. (4) contains no Lennard-Jones term here.

Table 2 and Fig. 3 summarize the checks. For a single molecule, the forces of the MD engine

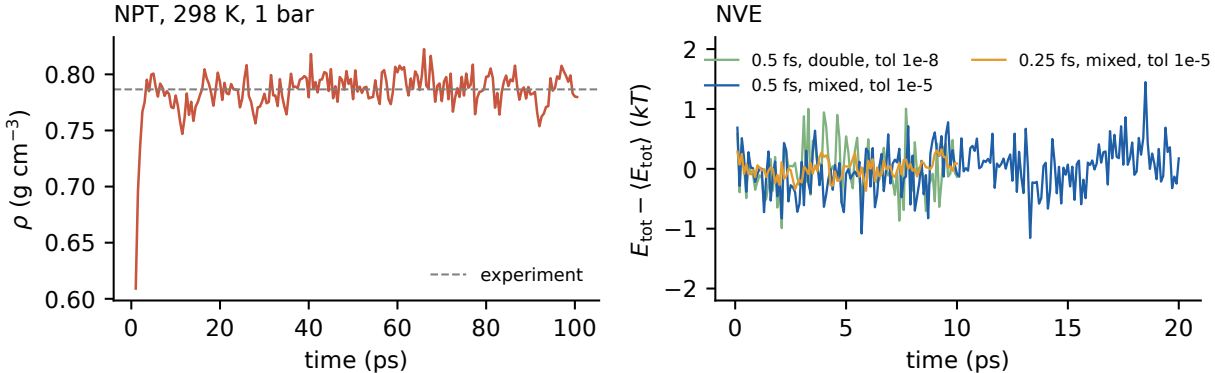


Figure 3: 216 flexible methanols (1,296 atoms). Left: density under NPT (298 K, 1 bar) from a dilute start at 0.55 g cm^{-3} (GAFF Lennard-Jones); dashed: experiment. Right: total energy in NVE from the equilibrated state, relative to its mean, in units of kT .

Table 2: Checks of flexible-molecule MD for 216 methanols (tol.: `dipole_scf_tol`; NVE drifts per degree of freedom).

Check	Result
One molecule in a 4 nm box vs the gas-phase model (double precision, 1.8 nm cutoff)	max. force difference $0.24 \text{ kJ mol}^{-1} \text{ nm}^{-1}$ for an RMS force of 939 (2.6×10^{-4})
NPT, 298 K, 1 bar, 100 ps from 0.55 g cm^{-3}	$\rho = 0.789 \pm 0.002 \text{ g cm}^{-3}$ over the last 50 ps (standard deviation 0.012; experiment 0.7866); 0.77 reached after 3 ps
Temperatures (NPT, last 50 ps)	296.0 K centre of mass, 295.9 K internal (target 298 K)
NVE, 0.5 fs, mixed precision, tol. 10^{-5} , 20 ps	drift $+0.004 \text{ kT/ns}$ per degree of freedom; RMS fluctuation 0.37 kT (whole system)
NVE, 0.25 fs, mixed precision, tol. 10^{-5} , 10 ps	drift $+0.003 \text{ kT/ns}$; fluctuation 0.14 kT
NVE, 0.5 fs, double precision, tol. 10^{-8} , 10 ps	drift -0.0005 kT/ns ; fluctuation 0.40 kT
Speed (NPT, Monte Carlo move every 25 steps, one GPU)	52 ns/day (0.83 ms per step)

(PME electrostatics, bonded terms) agree with the gradient of the gas-phase model the bonded terms were fitted with, up to the interaction with periodic images and the PME error. A box started at 0.55 g cm^{-3} reaches 0.77 g cm^{-3} within 3 ps (the neighbour lists are rebuilt on the way) and fluctuates around 0.789 g cm^{-3} with GAFF Lennard-Jones, against 0.7866 in experiment [36]. Translational and internal temperatures agree, so the Langevin thermostat equilibrates the fast bond vibrations and the molecular motion alike. In NVE the total energy is conserved: the drift is below 0.005 kT/ns per degree of freedom in all three runs, which is the precision of these 10–20 ps estimates; the fluctuation drops from 0.37 to 0.14 kT when the time step is halved, and double precision with a 10^{-8} dipole tolerance does not change it, so the remaining error comes from the integrator, not from the induction solver. At 0.5 fs the engine runs 52 ns/day for this system.

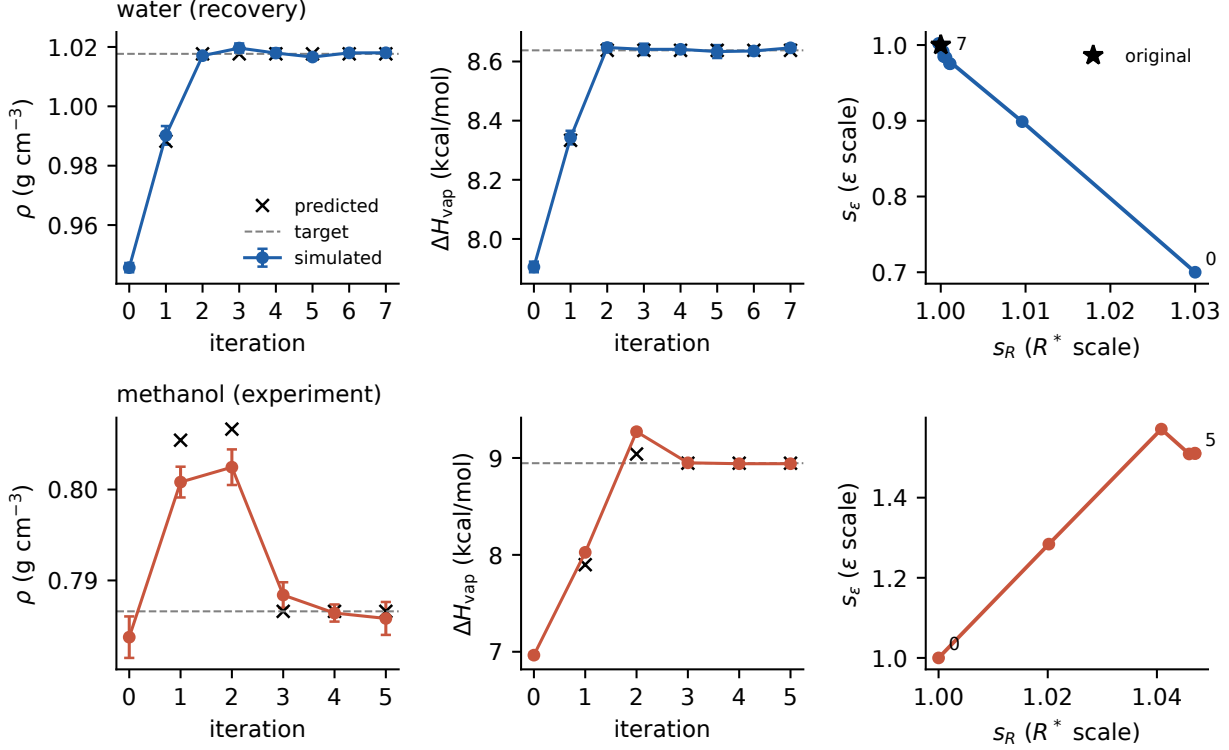


Figure 4: Gauss-Newton fits of two Lennard-Jones scales to the liquid density and heat of vaporization. Top: parameter recovery for 512 rigid pGM waters, from $s_R = 1.03$, $s_\epsilon = 0.70$ to the model’s own properties at $s_R = s_\epsilon = 1$. Bottom: 216 flexible methanols from GAFF Lennard-Jones to experiment. Circles: simulated averages (± 2 standard errors); crosses: values predicted from the previous iteration’s Jacobian; dashed: targets. Right: path of the parameters.

6.2 Lennard-Jones parameters from liquid properties

We fitted Lennard-Jones parameters to the density ρ and the heat of vaporization ΔH_{vap} at 298 K and 1 bar with `scripts/fit_liquid.py`. The parameters are two global scales, $\theta = (\ln s_R, \ln s_\epsilon)$, with $R_i^* \rightarrow s_R R_i^*$ and $\epsilon_i \rightarrow s_\epsilon \epsilon_i$ for every atom. Each iteration is one NPT simulation (0.9 nm cutoff with the Lennard-Jones tail correction, PME spacing 0.08 nm and order 6, `dipole.scf_tol` = 10^{-5} , mixed precision, Langevin $\gamma = 1$ ps $^{-1}$, Monte Carlo volume moves every 25 steps): 20 ps of equilibration (100 ps in the first iteration), then 200 ps of production with a frame every 0.2 ps. For each of the 1,000 frames the script evaluates U , V and $\partial U / \partial \theta$ at the converged dipoles; Eq. (6) gives the Jacobian $\mathbf{J}$ of $(\rho, \Delta H_{\text{vap}})$, and the Gauss-Newton step $\Delta \theta = -(\tilde{\mathbf{J}}^T \tilde{\mathbf{J}} + 10^{-3} \mathbf{I})^{-1} \tilde{\mathbf{J}}^T \tilde{\mathbf{r}}$, with residuals and Jacobian scaled by 0.005 g cm $^{-3}$ and 0.05 kcal/mol, is clipped to $|\Delta \ln s_R| \leq 0.02$ and $|\Delta \ln s_\epsilon| \leq 0.25$. The next simulation starts from the last configuration and tests the linear prediction $\mathbf{y} + \mathbf{J} \Delta \theta$. Standard errors are from five blocks. As a check that the per-frame derivatives belong to the sampled energy, the energy recomputed from the parameters at every frame equals the integrator’s potential energy to 0.03 kJ/mol for water and 0.01 kJ/mol for methanol (of totals of -2×10^6 and -1.7×10^5 kJ/mol).

Water: parameter recovery. For 512 rigid pGM waters (1 fs time step) the targets were the model’s own values at its original parameters from a 400 ps run, $\rho = 1.0177 \pm 0.0009$ g cm $^{-3}$ and

$\Delta H_{\text{vap}} = 8.638 \pm 0.008$ kcal/mol (U_{gas} is the exact energy of the rigid monomer). Starting from $s_R = 1.03$ and $s_\varepsilon = 0.70$ ($\rho = 0.946$, $\Delta H_{\text{vap}} = 7.905$), the fit reached the targets within their statistical errors in two iterations (Fig. 4, top): the second iteration predicted (1.0177, 8.638) and gave $(1.0171 \pm 0.0003, 8.647 \pm 0.007)$. Over iterations 2–7 the parameters stayed at $s_R = 1.0005 \pm 0.0005$ and $s_\varepsilon = 0.985 \pm 0.009$ (mean and standard deviation), consistent with the original values and with the precision expected from the Jacobian and the standard errors (0.0006 in $\ln s_R$ and 0.014 in $\ln s_\varepsilon$, correlation -0.99). The two targets constrain ε only weakly for water: both respond to ε through nearly the same combination as to R^* (the scaled Jacobian has condition number 170), and both derivatives with respect to $\ln s_\varepsilon$ are negative ($\partial\rho/\partial\ln s_\varepsilon = -0.13$ g cm $^{-3}$, $\partial\Delta H_{\text{vap}}/\partial\ln s_\varepsilon = -2.7$ kcal/mol) because the Lennard-Jones energy of liquid water is repulsive overall (+690 kcal/mol for the 512-water configuration of Table 1). An iteration took 200 s on one GPU. The original parameters of this model give a heat of vaporization 1.9 kcal/mol below experiment (10.52 kcal/mol [37]) at a density 2% above experiment. A linear extrapolation from the Jacobian puts both experimental values at $s_R \approx 1.08$ and $s_\varepsilon \approx 0.08$, far outside the range where two global Lennard-Jones scales are a sensible model, so a fit of this water model to experiment needs other parameters (charges, polarizabilities or the form of the repulsion) to move as well.

Methanol: fit to experiment. For 216 flexible methanols (0.5 fs time step; bonded terms of Section 6.1) the targets were the experimental $\rho = 0.7866$ g cm $^{-3}$ [36] and $\Delta H_{\text{vap}} = 37.43$ kJ/mol = 8.946 kcal/mol [38]. $\langle U_{\text{gas}} \rangle$ comes from gas-phase Langevin dynamics with the same bonded and pGM model (16 replicas of 100 ps, ± 0.09 kJ/mol), shifted by the difference between the MD engine and the gas-phase model for one molecule in a large box. With GAFF Lennard-Jones parameters the model gives $\rho = 0.7838 \pm 0.0011$ g cm $^{-3}$, close to experiment, but $\Delta H_{\text{vap}} = 6.965 \pm 0.014$ kcal/mol, 2 kcal/mol too low. Here ε is well determined: $\partial\Delta H_{\text{vap}}/\partial\ln s_\varepsilon = +4.4$ to $+7.4$ kcal/mol, positive because in liquid methanol the Lennard-Jones energy is attractive overall (-2.5×10^3 kJ/mol for 216 molecules). The fit reached both targets in four iterations (Fig. 4, bottom). At $s_R = 1.047$ and $s_\varepsilon = 1.510$ the fourth iteration gave $\rho = 0.7864 \pm 0.0005$ g cm $^{-3}$ and $\Delta H_{\text{vap}} = 8.941 \pm 0.010$ kcal/mol, and the fifth, after a step below 10^{-3} , 0.7858 ± 0.0009 and 8.941 ± 0.011 . The first two steps were large ($\Delta\ln s_\varepsilon = 0.25$ and 0.20 , with $\Delta\ln s_R$ at its limit of 0.02), and their linear predictions missed by 0.004–0.005 g cm $^{-3}$ in density and 0.13–0.23 kcal/mol in ΔH_{vap} ; once the steps were small, predictions and simulations agreed within 0.002 g cm $^{-3}$ and 0.005 kcal/mol, comparable to the statistical errors of the two simulations involved. Unlike for water, the Jacobian is well conditioned (condition number 2.4), and the standard errors translate into uncertainties of 0.06% in s_R and 0.16% in s_ε . The fit raises the GAFF well depths by 51% and the radii by 4.7%: GAFF Lennard-Jones, developed for fixed charges, underbinds liquid methanol by 2 kcal/mol when combined with pGM electrostatics fitted to gas-phase potentials. This is the kind of van der Waals parameterization that Section 7 is meant for. An iteration took 8–14 min on one GPU (23 min for the first, which includes 100 ps of equilibration).

7 Parameterization workflows

The repository contains two step-by-step guides (`docs/howto_bonded.md` and `docs/howto_vdw.md`) and example scripts for the two parameterization tasks that come next in the pGM effort: bonded terms for flexible molecules, and van der Waals parameters.

Bonded terms (bonds, angles, torsions, couplings).

1. *Reference data.* Topology and conformers from a SMILES string (RDKit); the MACE-OFF minimum; Langevin MD with MACE-OFF at 500 K (training) and 298 K (test) and relaxed torsion scans; DFT energies, forces and dipoles for the sampled frames (Slurm arrays; only missing frames are resubmitted); pGM charges and covalent dipoles fitted to the B3LYP/aug-cc-pVTZ electrostatic potential with PyRESP and polarizabilities and widths from the pGM table, following the standard pGM protocol [5, 7]; Lennard-Jones from GAFF [27] as a start.
2. *Fit.* `python examples/fit_bonded_template.py <name> --families paper` fits the chosen families with the all-pair pGM model of Eq. (4), reports energy and force errors on the 298 K frames, and writes a `FlexibleTemplate`. Term families are a registry; a new one is a JAX function of internal coordinates, and the test suite checks its derivatives automatically.
3. *Check in MD.* `examples/run_flexible_liquid.py` builds a box and runs NVT and NPT; the checks of Section 6.1 (forces against the gas-phase model, NVE conservation, internal temperature, density) apply to every new template.

Van der Waals parameters.

1. *Liquid properties.* `scripts/fit_liquid.py` implements Section 6.2. The map from the optimization variables to the parameter table (`ParamMap`) is the only place that decides what is fitted: two global scales in the demonstration, or one R^* and one ε scale per atom type (`--params type`; with more parameters than targets the Gauss–Newton step is the minimum-norm step, so more targets or a prior are needed). Because $\partial U/\partial\theta$ comes from `jax.value_and_grad` through that map, changing the parameterization needs no new derivative code. Further targets (other temperatures, other liquids, fluctuation properties) add rows to the Jacobian.
2. *Gas-phase data.* Dimer and cluster energies, or SAPT-like electrostatic and induction components (`elec_decomposition`), are fitted with the gas-phase model and `jax.grad`; a combined objective is the sum of the gas-phase loss and the liquid residuals.
3. *Checks.* The predicted-versus-simulated comparison of each iteration, longer runs with the final parameters, and the tests of the MD engine.

8 Limitations and outlook

The MD engine implements Lennard-Jones only; other van der Waals forms (for example repulsion from overlapping Gaussian densities) need their own energy channel, real-space kernels and tail corrections. Flexible molecules have no bond constraints yet, so the time step is 0.5 fs, and bonded terms can be exported to MD only if they were fitted with the engine’s nonbonded model (pGM with all pairs, no charge flux). The ensemble gradient of $\langle U_{\text{gas}} \rangle$ is not yet implemented for molecules with intramolecular Lennard-Jones pairs. For boxes of about 10^4 atoms and more, `pmemd-pgm`’s hand-written CUDA kernels remain faster (69 against 44 ns/day for 12k atoms), and pGM-JAX runs on a single GPU. Trajectories are not differentiated; derivatives of ensemble averages carry the statistical error of the simulation. Fitted parameters are stored as JSON; writing them into Amber `prmtop` files is not implemented yet.

Natural next steps are per-type Lennard-Jones fits over several liquids at once, with gas-phase dimer data as a second objective; derivatives of hydration and solvation free energies, which need the same per-frame $\partial U/\partial\theta$ within free-energy estimators; and bonded parameters for the pGM small-molecule library, run through the same MD engine.

Code availability

pGM-JAX is available at [repository URL] under [license]. The repository contains the tests, the Amber validation inputs, the benchmark scripts, the fitting scripts and the data behind every figure of this paper (paper/).

Acknowledgments

[To be added.]

References

- [1] Jay W. Ponder, Chuanjie Wu, Pengyu Ren, Vijay S. Pande, John D. Chodera, Michael J. Schnieders, Imran Haque, David L. Mobley, Daniel S. Lambrecht, Robert A. DiStasio, Jr., Martin Head-Gordon, Gary N. I. Clark, Margaret E. Johnson, and Teresa Head-Gordon. Current status of the AMOEBA polarizable force field. *J. Phys. Chem. B*, 114(8):2549–2564, 2010. doi: 10.1021/jp910674d.
- [2] Pengyu Ren and Jay W. Ponder. Polarizable atomic multipole water model for molecular mechanics simulation. *J. Phys. Chem. B*, 107(24):5933–5947, 2003. doi: 10.1021/jp027815+.
- [3] B. T. Thole. Molecular polarizabilities calculated with a modified dipole interaction. *Chem. Phys.*, 59(3):341–350, 1981. doi: 10.1016/0301-0104(81)85176-2.
- [4] Dennis Elking, Tom Darden, and Robert J. Woods. Gaussian induced dipole polarization model. *J. Comput. Chem.*, 28(7):1261–1274, 2007. doi: 10.1002/jcc.20574.
- [5] Junmei Wang, Piotr Cieplak, Ray Luo, and Yong Duan. Development of polarizable Gaussian model for molecular mechanical calculations I: Atomic polarizability parameterization to reproduce ab initio anisotropy. *J. Chem. Theory Comput.*, 15(2):1146–1158, 2019. doi: 10.1021/acs.jctc.8b00603.
- [6] Haixin Wei, Ruxi Qi, Junmei Wang, Piotr Cieplak, Yong Duan, and Ray Luo. Efficient formulation of polarizable Gaussian multipole electrostatics for biomolecular simulations. *J. Chem. Phys.*, 153(11):114116, 2020. doi: 10.1063/5.0019560.
- [7] Shiji Zhao, Haixin Wei, Piotr Cieplak, Yong Duan, and Ray Luo. PyRESP: A program for electrostatic parameterizations of additive and induced dipole polarizable force fields. *J. Chem. Theory Comput.*, 18(6):3654–3670, 2022. doi: 10.1021/acs.jctc.2c00230.
- [8] Yong Duan, Taoyu Niu, Junmei Wang, Piotr Cieplak, and Ray Luo. PCMRESP: A method for polarizable force field parameter development and transferability of the polarizable Gaussian multipole models across multiple solvents. *J. Chem. Theory Comput.*, 20(7):2820–2829, 2024. doi: 10.1021/acs.jctc.4c00064.
- [9] Shiji Zhao, Piotr Cieplak, Yong Duan, and Ray Luo. Transferability of the electrostatic parameters of the polarizable Gaussian multipole model. *J. Chem. Theory Comput.*, 19(3): 924–941, 2023. doi: 10.1021/acs.jctc.2c01048.

- [10] Haixin Wei, Piotr Cieplak, Yong Duan, and Ray Luo. Stress tensor and constant pressure simulation for polarizable Gaussian multipole model. *J. Chem. Phys.*, 156(11):114114, 2022. doi: 10.1063/5.0082548.
- [11] Zhen Huang, Yongxian Wu, Yong Duan, and Ray Luo. Performance tuning of polarizable Gaussian multipole model in molecular dynamics simulations. *J. Chem. Theory Comput.*, 21(2):847–858, 2025. doi: 10.1021/acs.jctc.4c01368.
- [12] David A. Case, Hasan Metin Aktulga, Kellon Belfon, David S. Cerutti, G. Andrés Cisneros, Vinícius Wilian D. Cruzeiro, Negin Forouzesh, Timothy J. Giese, Andreas W. Götz, Holger Gohlke, Saeed Izadi, Koushik Kasavajhala, Mehmet C. Kaymak, Edward King, Tom Kurtzman, Tai-Sung Lee, Pengfei Li, Jian Liu, Tyler Luchko, Ray Luo, Madushanka Manathunga, Matias R. Machado, Hai Minh Nguyen, Kurt A. O’Hearn, Alexey V. Onufriev, Feng Pan, Sergio Pantano, Ruxi Qi, Ali Rahnamoun, Ali Risheh, Stephan Schott-Verdugo, Akhil Shajan, Jason Swails, Junmei Wang, Haixin Wei, Xiongwu Wu, Yongxian Wu, Shi Zhang, Shiji Zhao, Qiang Zhu, Thomas E. Cheatham, III, Daniel R. Roe, Adrian Roitberg, Carlos Simmerling, Darrin M. York, Maria C. Nagan, and Kenneth M. Merz, Jr. AmberTools. *J. Chem. Inf. Model.*, 63(20): 6183–6191, 2023. doi: 10.1021/acs.jcim.3c01153.
- [13] Yongxian Wu, Qiang Zhu, Zhen Huang, Piotr Cieplak, Yong Duan, and Ray Luo. Automated refinement of property-specific polarizable Gaussian multipole water models using Bayesian black-box optimization. *J. Chem. Theory Comput.*, 21(7):3563–3575, 2025. doi: 10.1021/acs.jctc.5c00039.
- [14] Lee-Ping Wang, Todd J. Martínez, and Vijay S. Pande. Building force fields: An automatic, systematic, and reproducible approach. *J. Phys. Chem. Lett.*, 5:1885–1891, 2014. doi: 10.1021/jz500737m.
- [15] Lee-Ping Wang, Teresa Head-Gordon, Jay W. Ponder, Pengyu Ren, John D. Chodera, Peter K. Eastman, Todd J. Martinez, and Vijay S. Pande. Systematic improvement of a classical molecular model of water. *J. Phys. Chem. B*, 117(34):9956–9972, 2013. doi: 10.1021/jp403802c.
- [16] James Bradbury, Roy Frostig, Peter Hawkins, Matthew James Johnson, Yash Katariya, Chris Leary, Dougal Maclaurin, George Neca, Adam Paszke, Jake VanderPlas, Skye Wanderman-Milne, and Qiao Zhang. JAX: composable transformations of Python+NumPy programs. <http://github.com/jax-ml/jax>, 2018.
- [17] Samuel Schoenholz and Ekin Dogus Cubuk. JAX MD: A framework for differentiable physics. In *Advances in Neural Information Processing Systems*, volume 33, pages 11428–11441, 2020. URL <https://proceedings.neurips.cc/paper/2020/hash/83d3d4b6c9579515e1679aca8cbc8033-Abstract.html>.
- [18] Stefan Doerr, Maciej Majewski, Adrià Pérez, Andreas Krämer, Cecilia Clementi, Frank Noé, Toni Giorgino, and Gianni De Fabritiis. TorchMD: A deep learning framework for molecular simulations. *J. Chem. Theory Comput.*, 17(4):2355–2363, 2021. doi: 10.1021/acs.jctc.0c01343.
- [19] Stephan Thaler and Julija Zavadlav. Learning neural network potentials from experimental data via differentiable trajectory reweighting. *Nat. Commun.*, 12:6884, 2021. doi: 10.1038/s41467-021-27241-4.

- [20] Xinyan Wang, Jichen Li, Lan Yang, Feiyang Chen, Yingze Wang, Junhan Chang, Junmin Chen, Wei Feng, Linfeng Zhang, and Kuang Yu. DMFF: An open-source automatic differentiable platform for molecular force field development and molecular dynamics simulation. *J. Chem. Theory Comput.*, 19(17):5897–5909, 2023. doi: 10.1021/acs.jctc.2c01297.
- [21] Mathieu Blondel, Quentin Berthet, Marco Cuturi, Roy Frostig, Stephan Hoyer, Felipe Llinares-López, Fabian Pedregosa, and Jean-Philippe Vert. Efficient and modular implicit differentiation. In *Advances in Neural Information Processing Systems*, volume 35, pages 5230–5242, 2022. URL https://proceedings.neurips.cc/paper_files/paper/2022/hash/228b9279ecf9bbafe582406850c57115-Abstract-Conference.html.
- [22] Ulrich Essmann, Lalith Perera, Max L. Berkowitz, Tom Darden, Hsing Lee, and Lee G. Pedersen. A smooth particle mesh Ewald method. *J. Chem. Phys.*, 103(19):8577–8593, 1995. doi: 10.1063/1.470117.
- [23] Thomas F. Miller, III, M. Eleftheriou, P. Pattnaik, A. Ndirango, D. Newns, and G. J. Martyna. Symplectic quaternion scheme for biophysical molecular dynamics. *J. Chem. Phys.*, 116(20): 8649–8659, 2002. doi: 10.1063/1.1473654.
- [24] Benedict Leimkuhler and Charles Matthews. Rational construction of stochastic numerical methods for molecular sampling. *Appl. Math. Res. Express*, 2013(1):34–56, 2013. doi: 10.1093/amrx/abs010.
- [25] Kim-Hung Chow and David M. Ferguson. Isothermal-isobaric molecular dynamics simulations with Monte Carlo volume sampling. *Comput. Phys. Commun.*, 91(1–3):283–289, 1995. doi: 10.1016/0010-4655(95)00059-O.
- [26] Johan Åqvist, Petra Wennerström, Martin Nervall, Sinisa Bjelic, and Bjørn O. Brandsdal. Molecular dynamics simulations of water and biomolecules with a Monte Carlo constant pressure algorithm. *Chem. Phys. Lett.*, 384(4–6):288–294, 2004. doi: 10.1016/j.cplett.2003.12.039.
- [27] Junmei Wang, Romain M. Wolf, James W. Caldwell, Peter A. Kollman, and David A. Case. Development and testing of a general amber force field. *J. Comput. Chem.*, 25(9):1157–1174, 2004. doi: 10.1002/jcc.20035.
- [28] Pnina Dauber-Osguthorpe and A. T. Hagler. Biomolecular force fields: where have we been, where are we now, where do we need to go and how do we get there? *J. Comput.-Aided Mol. Des.*, 33(2):133–203, 2019. doi: 10.1007/s10822-018-0111-4.
- [29] Aalim S. Abdullah, Yingze Wang, Maximilian F. S. J. Menger, Selim Sami, and Teresa Head-Gordon. Improved treatment of 1–4 interactions in force fields for molecular dynamics simulations. *J. Chem. Theory Comput.*, 21(22):11669–11678, 2025. doi: 10.1021/acs.jctc.5c01458.
- [30] Dávid Péter Kovács, J. Harry Moore, Nicholas J. Browning, Ilyes Batatia, Joshua T. Horton, Yixuan Pu, Venkat Kapil, William C. Witt, Ioan-Bogdan Magdău, Daniel J. Cole, and Gábor Csányi. MACE-OFF: Short-range transferable machine learning force fields for organic molecules. *J. Am. Chem. Soc.*, 147(21):17598–17611, 2025. doi: 10.1021/jacs.4c07099.
- [31] Narbe Mardirossian and Martin Head-Gordon. ω B97M-V: A combinatorially optimized, range-separated hybrid, meta-GGA density functional with VV10 nonlocal correlation. *J. Chem. Phys.*, 144(21):214110, 2016. doi: 10.1063/1.4952647.

- [32] Asim Najibi and Lars Goerigk. The nonlocal kernel in van der Waals density functionals as an additive correction: An extensive analysis with special emphasis on the B97M-V and ω B97M-V approaches. *J. Chem. Theory Comput.*, 14(11):5725–5738, 2018. doi: 10.1021/acs.jctc.8b00842.
- [33] Stefan Grimme, Stephan Ehrlich, and Lars Goerigk. Effect of the damping function in dispersion corrected density functional theory. *J. Comput. Chem.*, 32(7):1456–1465, 2011. doi: 10.1002/jcc.21759.
- [34] Dmitrij Rappoport and Filipp Furche. Property-optimized Gaussian basis sets for molecular response calculations. *J. Chem. Phys.*, 133(13):134105, 2010. doi: 10.1063/1.3484283.
- [35] Daniel G. A. Smith, Lori A. Burns, Andrew C. Simmonett, Robert M. Parrish, Matthew C. Schieber, Raimondas Galvelis, Peter Kraus, Holger Kruse, Roberto Di Remigio, Asem Alenaizan, Andrew M. James, Susi Lehtola, Jonathon P. Misiewicz, Maximilian Scheurer, Robert A. Shaw, Jeffrey B. Schriber, Yi Xie, Zachary L. Glick, Dominic A. Sirianni, Joseph Senan O’Brien, Jonathan M. Waldrop, Ashutosh Kumar, Edward G. Hohenstein, Benjamin P. Pritchard, Bernard R. Brooks, Henry F. Schaefer, III, Alexander Yu. Sokolov, Konrad Patkowski, A. Eugene DePrince, III, Uğur Bozkaya, Rollin A. King, Francesco A. Evangelista, Justin M. Turney, T. Daniel Crawford, and C. David Sherrill. Psi4 1.4: Open-source software for high-throughput quantum chemistry. *J. Chem. Phys.*, 152(18):184108, 2020. doi: 10.1063/5.0006002.
- [36] R. Srivastava and B. D. Smith. Evaluation of binary excess volume data for the methanol+hydrocarbon systems. *J. Phys. Chem. Ref. Data*, 16:209–218, 1987. doi: 10.1063/1.555780.
- [37] Hans W. Horn, William C. Swope, Jed W. Pitera, Jeffrey D. Madura, Thomas J. Dick, Greg L. Hura, and Teresa Head-Gordon. Development of an improved four-site water model for biomolecular simulations: TIP4P-Ew. *J. Chem. Phys.*, 120(20):9665–9678, 2004. doi: 10.1063/1.1683075.
- [38] Jiří Polák and G. C. Benson. Enthalpies of vaporization of some aliphatic alcohols. *J. Chem. Thermodyn.*, 3(2):235–242, 1971. doi: 10.1016/S0021-9614(71)80108-8.